Task 3 Report

CI/CD Automation for Dockerized Application using GitHub Actions

Submitted by:

Namratha T G

1. Introduction

Continuous Integration (CI) is a DevOps practice where code changes are automatically built and tested whenever they are pushed to a repository.

In this task, a Dockerized static website application was automated using GitHub Actions. The pipeline automatically builds and pushes a Docker image to Docker Hub whenever code is pushed to the main branch.

This eliminates the need for manual Docker build and push operations.

2. Objective

The objective of this task is to:

- Automate Docker image build process
- Push Docker image to Docker Hub automatically
- Implement CI using GitHub Actions
- Secure credentials using GitHub Secrets
- Validate the image locally

3. Tools and Technologies Used

- Git
- GitHub
- Docker
- Docker Hub
- GitHub Actions
- Nginx (Base Docker Image)

- Windows OS

4. Project Description

The project is a simple static portfolio website developed using HTML.

The application is containerized using Docker with the following Dockerfile:

```
FROM nginx:alpine
COPY . /usr/share/nginx/html
EXPOSE 80
```

This Dockerfile:

- Uses lightweight Nginx image
- Copies website files into container
- Exposes port 80

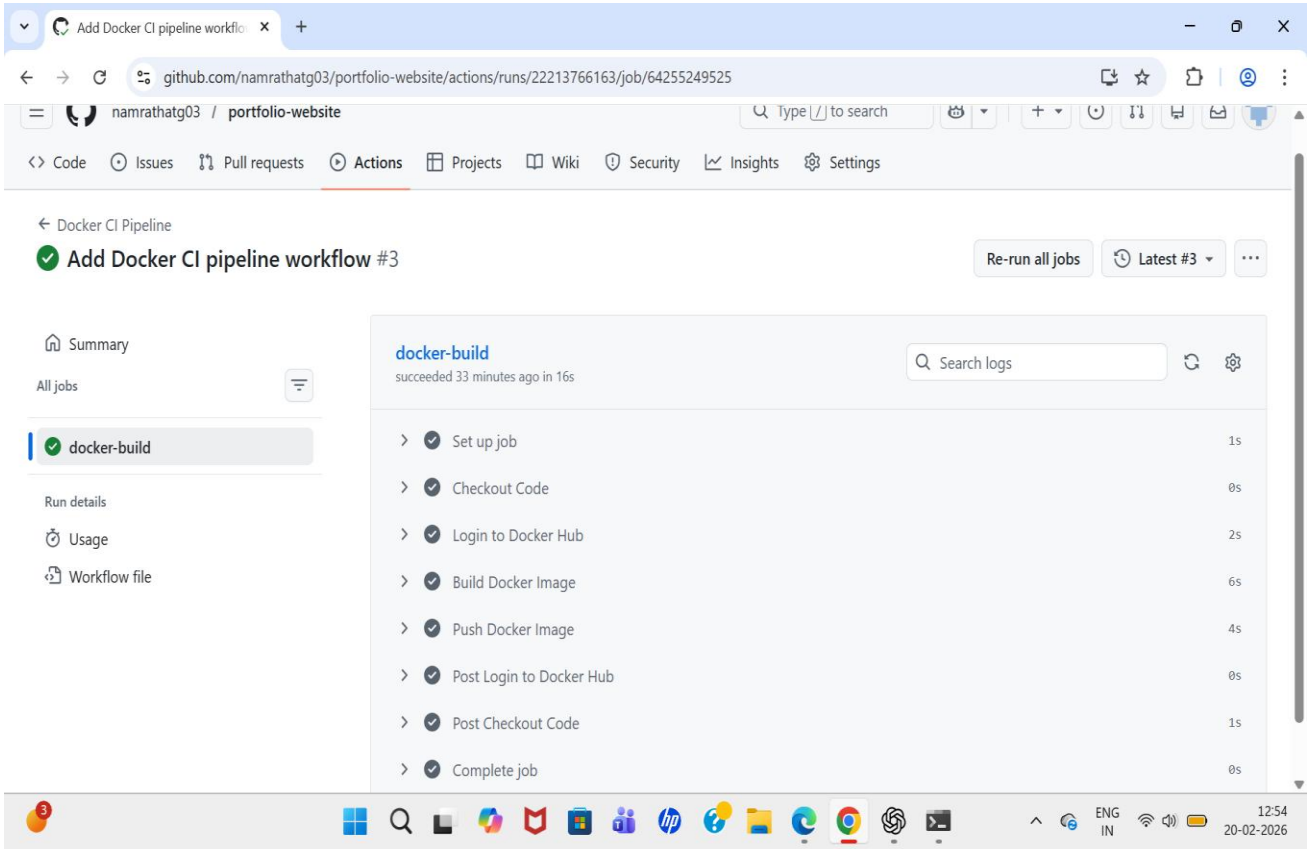

Figure 1.1: Dockerfile in GitHub repository

5. GitHub Actions Workflow Configuration

A workflow file was created at:

.github/workflows/docker-ci.yml

The workflow is triggered on every push to the main branch.

Workflow YAML:

```
name: Docker CI Pipeline
on:
  push:
    branches:
      - main

jobs:
  docker-build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Code
        uses: actions/checkout@v3

      - name: Login to Docker Hub
        uses: docker/login-action@v2
        with:
          username: ${{ secrets.DOCKER_USERNAME }}
          password: ${{ secrets.DOCKER_PASSWORD }}

      - name: Build Docker Image
        run: docker build -t namrathatg/portfolio-website:latest .
```

- name: Push Docker Image

  run: docker push namrathatg/portfolio-website:latest

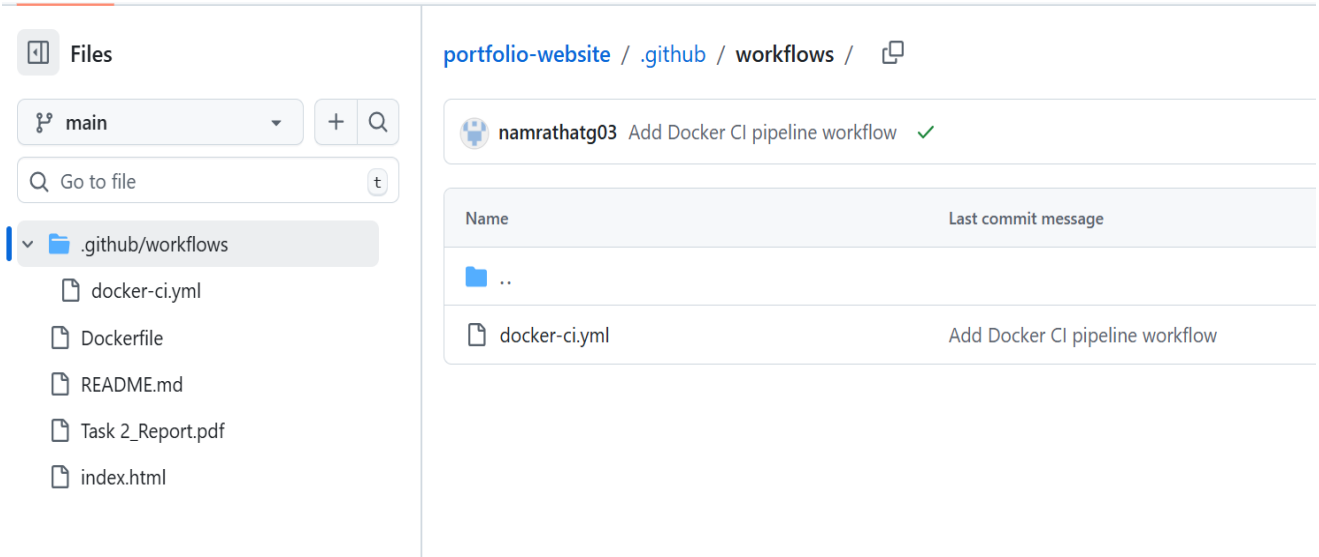

Figure 1.2: docker-ci.yml file in GitHub

6. GitHub Secrets Configuration

To securely authenticate with Docker Hub, the following secrets were configured:

- DOCKER_USERNAME
- DOCKER_PASSWORD (Docker Personal Access Token)

These secrets prevent exposing credentials inside the workflow file.

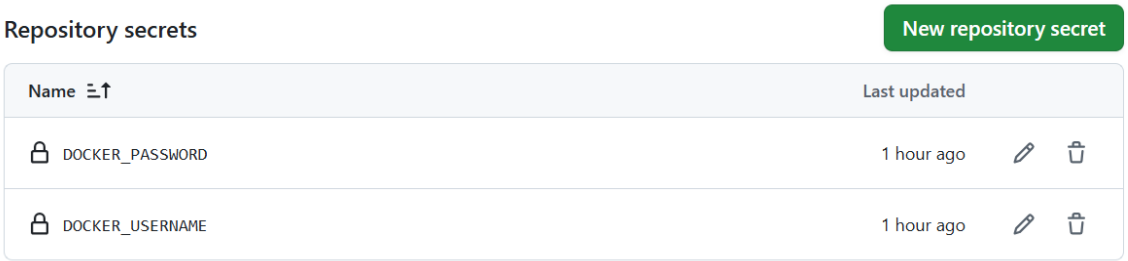

Figure 1.3: Repository secrets

7. CI Pipeline Execution

When the workflow file was committed, GitHub Actions automatically executed the pipeline.

Pipeline steps:

1. Code checkout
2. Docker login
3. Docker image build
4. Docker image push

The pipeline completed successfully.

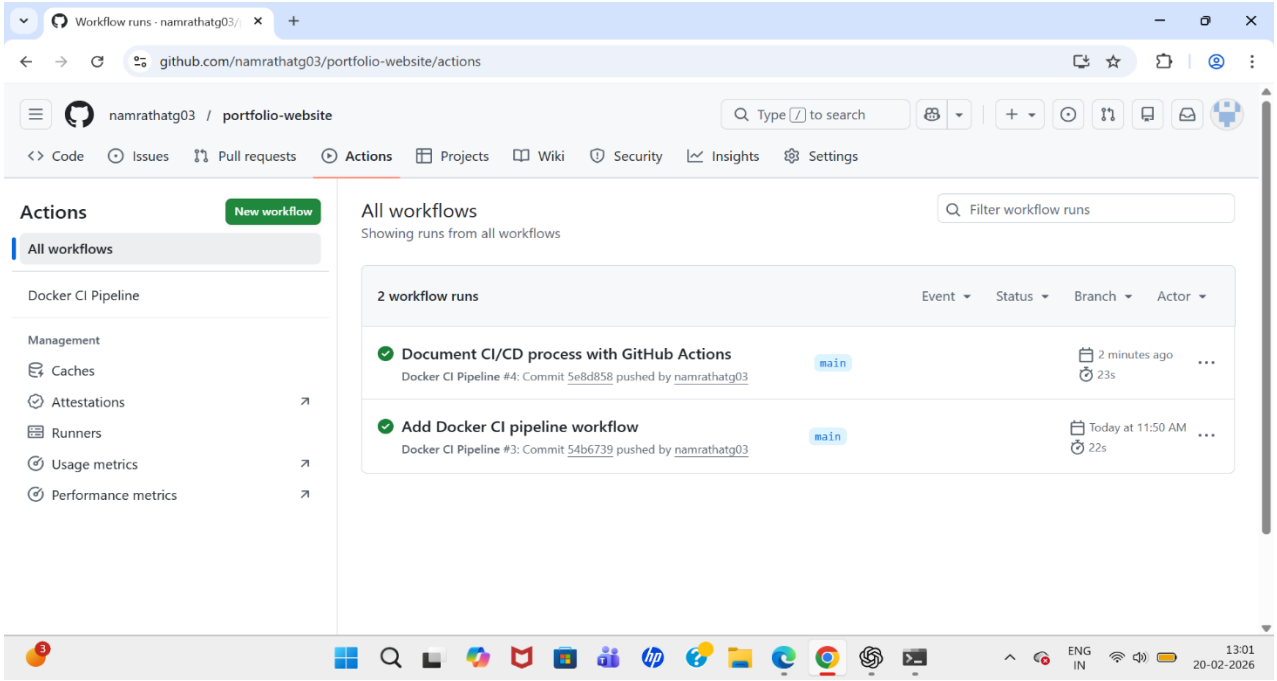

Figure 1.4: Green successful GitHub Actions run

8. Docker Hub Verification

After successful execution, the image was automatically pushed to Docker Hub.

Docker Repository:

namrathatg/portfolio-website

The image tag:

latest

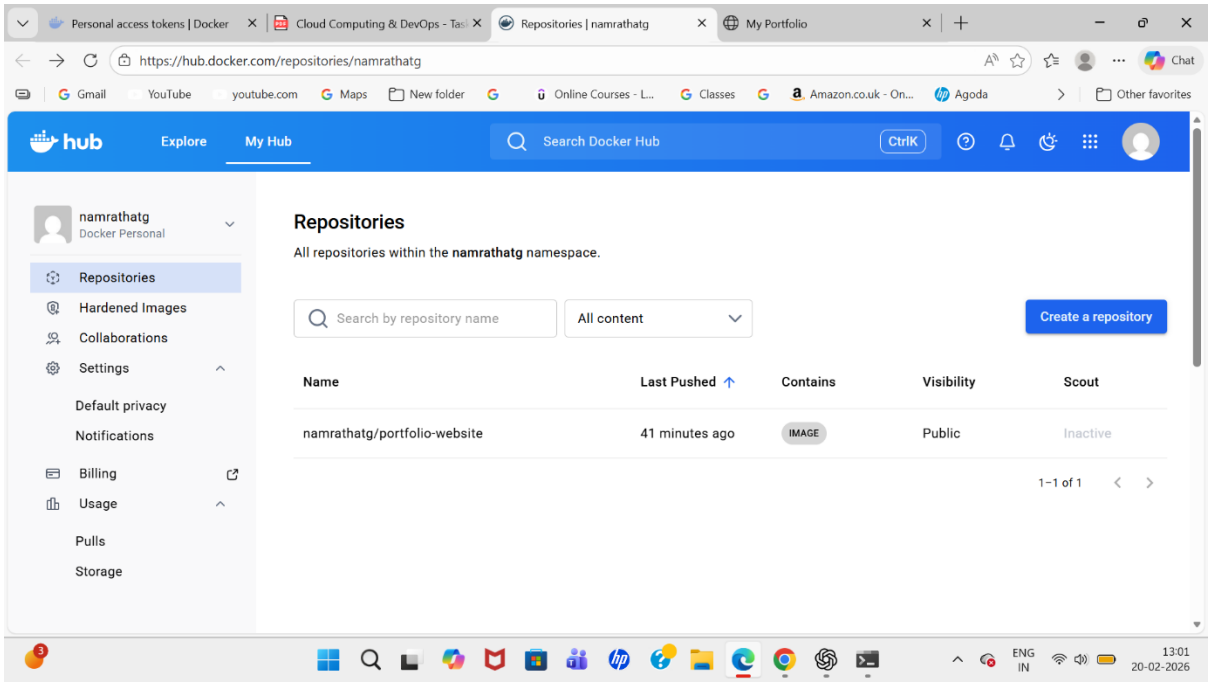

Figure 1.5: Docker Hub repository page

9. Image Validation (Local Testing)

The image was pulled and tested locally using:

```
docker pull namrathatg/portfolio-website:latest
docker run -d -p 8080:80 namrathatg/portfolio-website
```

The application was successfully accessed at:

http://localhost:8080

This confirms that the image works correctly.

```
C:\Users\Namratha TG>docker pull namrathatg/portfolio-website:latest
latest: Pulling from namrathatg/portfolio-website
51108c12d875: Pull complete
Digest: sha256:355bd97460607454669101a264362b8d7fa9ad785e3bcdf3d5d87f198f9dae98
Status: Downloaded newer image for namrathatg/portfolio-website:latest
docker.io/namrathatg/portfolio-website:latest

C:\Users\Namratha TG>docker run -d -p 8080:80 namrathatg/portfolio-website
5f3f823493e19360ce250a128aca741a6a6a987cf56fea72e184537519ad206c
```

Figure 1.6: Terminal showing docker pull and docker run commands

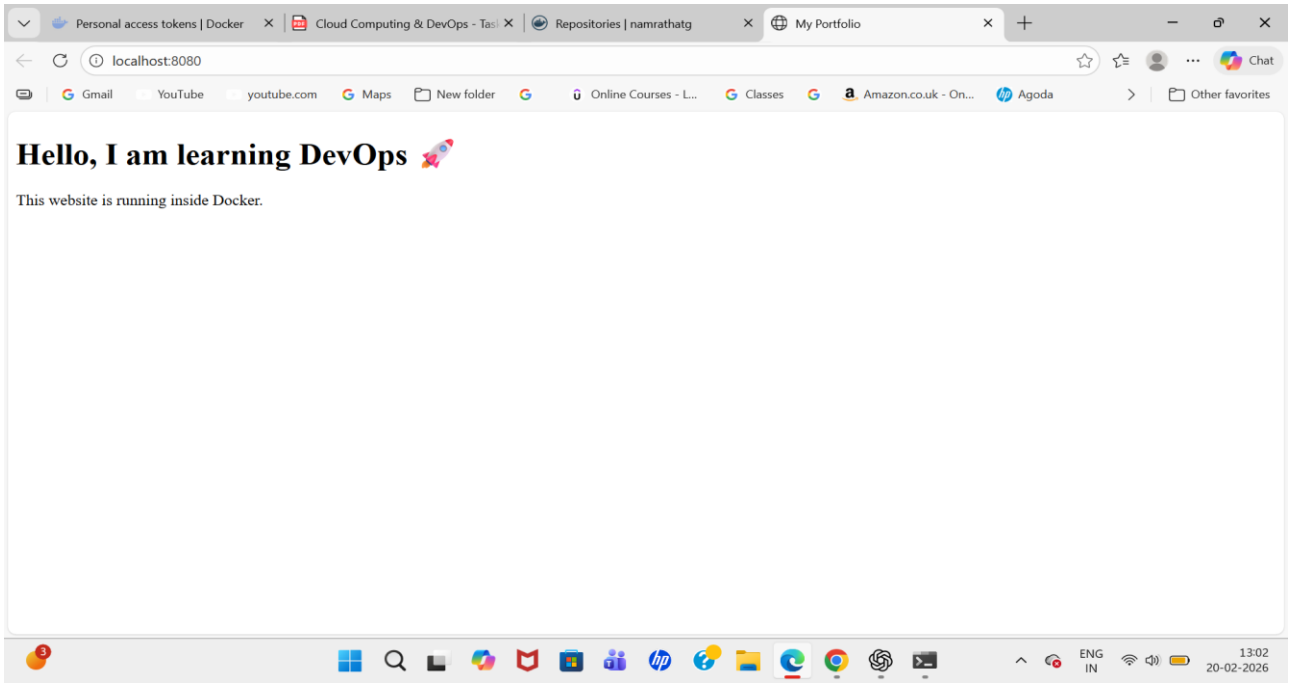

Figure 1.7: Browser showing website running on localhost:8080

10. Advantages of CI/CD Implementation

- Eliminates manual Docker build process
- Reduces human errors
- Saves time
- Ensures consistent deployment
- Provides automated integration workflow

- Follows real-world DevOps practices

11. Challenges Faced

During implementation, the following issues were encountered:

- Incorrect Docker authentication
- Insufficient access token scope
- Secret configuration errors

These were resolved by:

- Creating Docker Personal Access Token with Read/Write/Delete permissions
- Correctly configuring GitHub repository secrets
- Re-running the pipeline after corrections

12. Conclusion

In this task, CI/CD automation was successfully implemented using GitHub Actions.

The Dockerized website application is now automatically built and pushed to Docker Hub whenever changes are pushed to the repository.

This demonstrates practical implementation of Continuous Integration in a real DevOps environment.